ClusterWatch

Rajan Majithiya

October 2026

What’s inside

Chapter	You will learn
The story	The problem ClusterWatch solves, and what it does
How it is built	Architecture, the life of one health check, key design decisions
Node.js	The event loop in one picture, how Node is used here, commands
Docker	Images and containers, the Dockerfile step by step, measurements, commands
Kubernetes	The building blocks, three experiments, commands
Shipping it	The CI/CD pipeline and going live safely
How to run it	Three ways: Docker, plain Node.js, Kubernetes
Telling the story in an interview	30-second and 2-minute versions, five real bugs, expected questions

The story

2:07 a.m.

It is 2:07 in the morning. A payments service at a company that builds software for police and fire departments has stopped answering. Nobody knows yet. The servers are on, the dashboards are green, and the on-call engineer is asleep.

At 2:41 a dispatcher calls support: “Payments are failing.” Thirty-four minutes of outage, discovered by a customer.

That is the problem **ClusterWatch** solves.

What ClusterWatch does

ClusterWatch is a small monitoring platform, a mini Pingdom built for a team’s own services.

1. You **register a service** (a “target”): a name and a health URL, such as `https://payments/healthz`.
2. A **worker** checks every target on a schedule (every 5 to 3600 seconds) with a hard timeout.
3. After **3 failures in a row** it opens **one** alert. Not one per failed check, and not one per worker.
4. After **2 successes in a row** it **resolves the alert by itself**.
5. Engineers watch a **live dashboard**: what is up, what is down, how slow it is, and the history.

With ClusterWatch watching, the 2:07 outage opens an alert at 2:07:15, not 2:41.

Why it was built this way

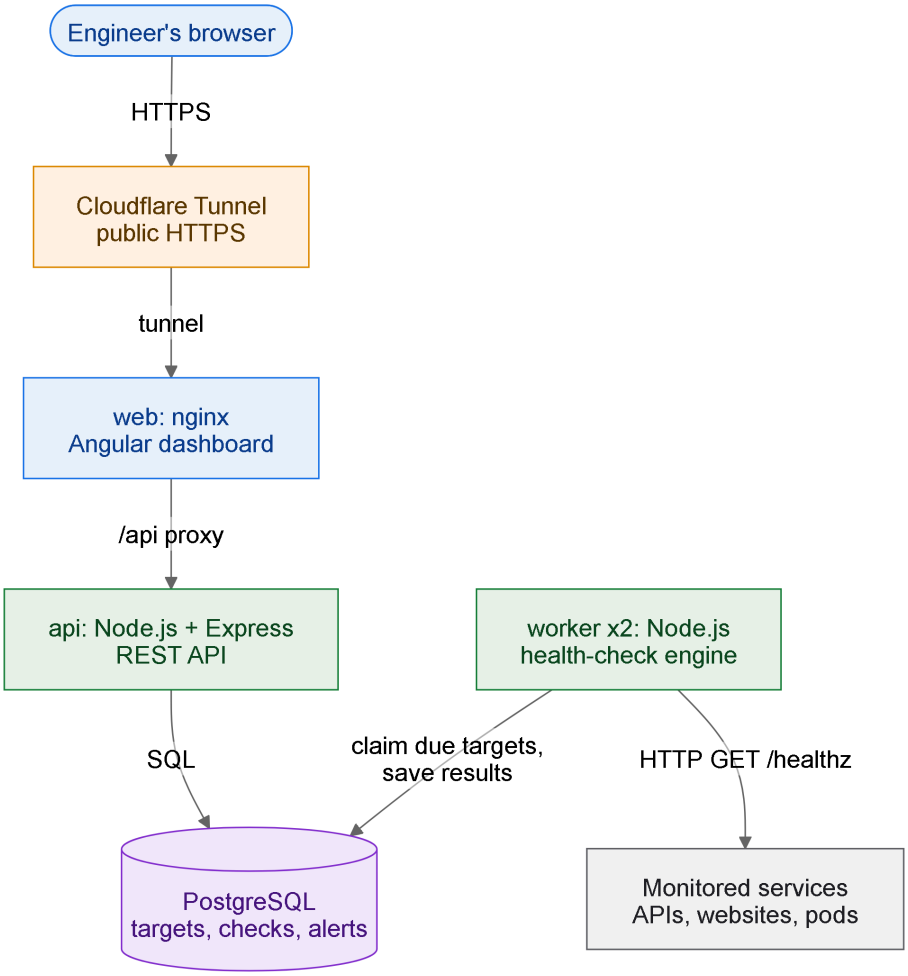
I built ClusterWatch to practise the exact stack of a cloud platform team: **Node.js** for the services, **PostgreSQL** for data, **Angular** for the dashboard, **Docker** to package it, **Kubernetes** to run it, and **GitHub Actions** to test and ship it.

I built it like production, not like a tutorial. After every feature I **broke it on purpose**: killed the database, shipped a wrong password, restarted it under load, attacked it with internal URLs. Then I fixed what broke. Most of what is worth telling in an interview came from those failures.

Measured result	Number
Automated tests (API + dashboard)	62 + 19, all passing in CI
Docker image, naive vs optimised	2.02 GB to 250 MB
Requests failed during a rolling restart under load	0 (23 without the fix)
Requests failed while a broken version was being deployed	0 of 45
Crash time on a wrong database password	65 s to 3 s (after a fix)
Post-deploy checks run over the public internet	25 of 25 passed

How it is built

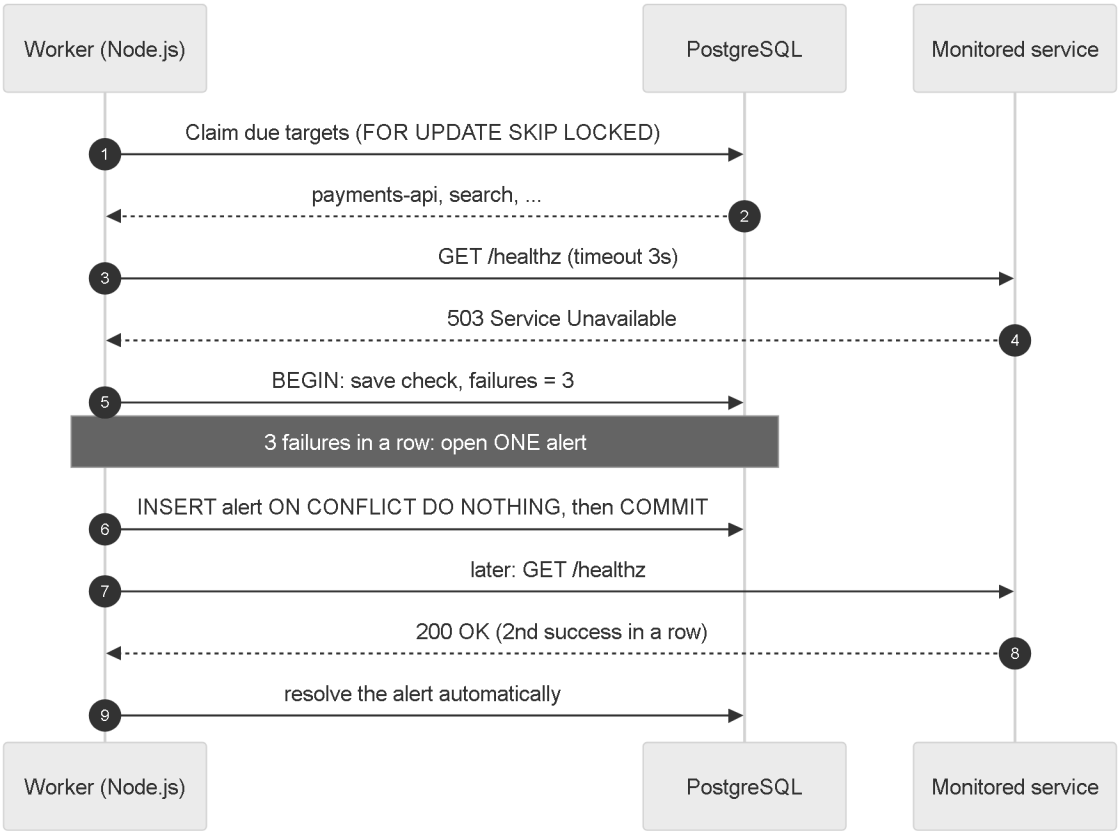
The big picture



Part	Technology	Job
web	Angular 22, served by nginx	The dashboard. nginx also forwards `/api` to the API, so the browser talks to one address (no CORS).
api	Node.js 24, Express 5	REST API: create and list targets, list and resolve alerts. Validates input, checks the admin token, rate-limits.
worker (×2)	Node.js 24	The engine. Claims due targets, checks them over HTTP, records results, opens and resolves alerts.
database	PostgreSQL 18	Targets, every check result, alerts. Schema changes are versioned migration files.
tunnel	Cloudflare	Gives a public HTTPS URL without opening any port on the network.

Why is the worker separate from the API? They scale differently: the API with the number of users, the worker with the number of targets. And a storm of slow health checks can never slow down the dashboard.

The life of one health check



Three ideas make this safe with any number of workers:

- `FOR UPDATE SKIP LOCKED`. Each worker locks the rows it claims, and other workers *skip* locked rows instead of waiting. Two workers never check the same target, with no coordinator.
- **One transaction.** The check result, the new failure count and the alert change are saved together or not at all.
- **A unique index** allows only one open alert per target. Even a bug cannot create duplicates.

The decisions that matter

Decision	Why
Every outbound check has a **timeout**	A service that never answers must not freeze the worker.
Hysteresis: 3 failures to open, 2 successes to close	A flapping service does not wake anyone up 40 times a night.
Database down → the API returns **503**, not 500	500 means “our bug” and pages on-call; 503 means “temporary, retry”.
Liveness checks the process, **readiness** checks the database	A database blip should stop traffic, not restart every pod.
Public **read**, **token** for writes	Anyone can look; only the admin can change things.
SSRF guard: no private or internal addresses, checked at connect time	A tool that fetches URLs must not be usable to scan the internal network.

Node.js

What Node.js is, in one picture

Node.js runs JavaScript on a server. Its trick is the **event loop**: one thread that never waits.

Think of **one waiter in a busy restaurant**. They take an order, hand it to the kitchen, and go straight to the next table. They don't stand at the kitchen door. When a dish is ready, the kitchen rings a bell and the waiter carries it out. One waiter serves fifty tables because they never wait.

In ClusterWatch, "the kitchen" is the network and the database. A worker sends 20 health checks at once and handles each answer as it arrives. That is why Node.js suits **I/O-heavy** work like APIs and monitoring.

The catch: if the waiter starts cooking (heavy CPU work, like resizing images), every table waits. So CPU-heavy work goes to worker threads or another service.

How Node.js is used here

Concept	Where in ClusterWatch
Express middleware: a pipeline every request flows through	`src/app.js`: request log, then JSON parser, rate limit, auth, routes, 404, error handler
async/await	Every database query and HTTP check
Central error handler	`src/errors.js` turns errors into 400 / 401 / 404 / 409 / 503 / 500
Graceful shutdown	On SIGTERM: stop taking requests, finish the current ones, close the DB pool, exit
Connection pool	Reuses database connections instead of opening one per request
Config from environment variables	The same code runs on a laptop, in Docker and in Kubernetes
Tests	Node's built-in test runner, with the same tests run against both an in-memory store and real Postgres

Node.js commands

Command	What it does
`npm ci`	Install the exact versions in `package-lock.json` (reproducible)
`npm run db`	Start a local PostgreSQL on port 5433 (no Docker needed)
`npm run dev`	Start the API on port 3000 and restart it when code changes
`npm run worker`	Start the health-check worker
`npm run fake-targets`	Start fake services that fail in realistic ways, on port 4000
`npm test`	Run all 62 API tests

Docker

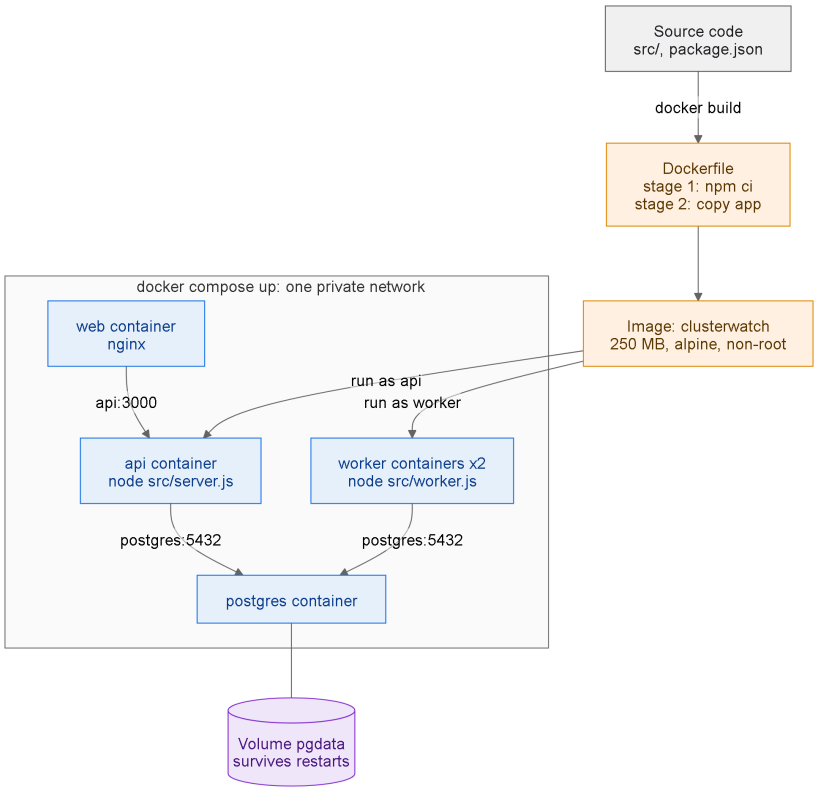
The problem Docker solves

“It works on my machine.” My laptop has Node 24 and Windows. The server has a different Node, different libraries, Linux. The same code behaves differently.

Docker packs the application **together with everything it needs** (the right Node version, the libraries, the files) into one sealed box called an **image**. The box runs the same everywhere.

- **Image**: the packed box, a frozen meal. Read-only. You build it once.
- **Container**: the box running, the meal heated up. You can run many containers from one image.
- **Dockerfile**: the recipe for building the image.
- **Volume**: a storage area outside the container, so data survives when containers are replaced.

How Docker is used here



One image, two programs. The API and the worker are the same image started with a different command. One build, one thing to test, one thing to ship.

The Dockerfile, step by step:

1. Start from `node:24-alpine`, a small Linux with Node (about 50 MB instead of about 1 GB).
2. Copy **only** `package.json` and `package-lock.json`, then run `npm ci --omit=dev`. Docker caches this step, so changing code later does not reinstall packages. Rebuilds take 9 seconds instead of minutes.
3. Start a **fresh second stage** and copy in only the app and its production packages. Nothing else comes along.
4. `USER 1000`: run as a normal user, not root.
5. `CMD ["node", "src/server.js"]`: the JSON form makes Node the main process, so it receives the stop signal and can shut down gracefully.

docker compose starts the whole system (database, API, two workers, dashboard) with one command. Containers find each other by name: the API connects to `postgres:5432`.

What I measured

	Naive Dockerfile	Final Dockerfile
Image size	2.02 GB	250 MB
Rebuild after a code change	7 minutes	9 seconds
Runs as	root	user 1000
Leaked my local database files into the image	yes	no (`.dockerignore`)
`docker stop`	killed after the timeout (exit 137)	clean shutdown (exit 0)

Docker commands

Command	What it does
`docker build -t clusterwatch:dev .`	Build the image from the Dockerfile
`docker image ls`	List images and their sizes
`docker run --rm -p 3000:3000 -e ADMIN_TOKEN=x clusterwatch:dev`	Run one container, publish port 3000
`docker compose up -d --build --wait`	Build and start everything, wait until healthy
`docker compose ps`	Status and health of every container
`docker compose logs -f worker`	Follow a service's logs
`docker compose exec -T postgres psql -U clusterwatch -d clusterwatch`	Run a command inside a container
`docker compose stop api`	Send SIGTERM, wait, then SIGKILL
`docker compose down`	Remove containers (add `-v` to delete the data volume too)

Kubernetes

The problem Kubernetes solves

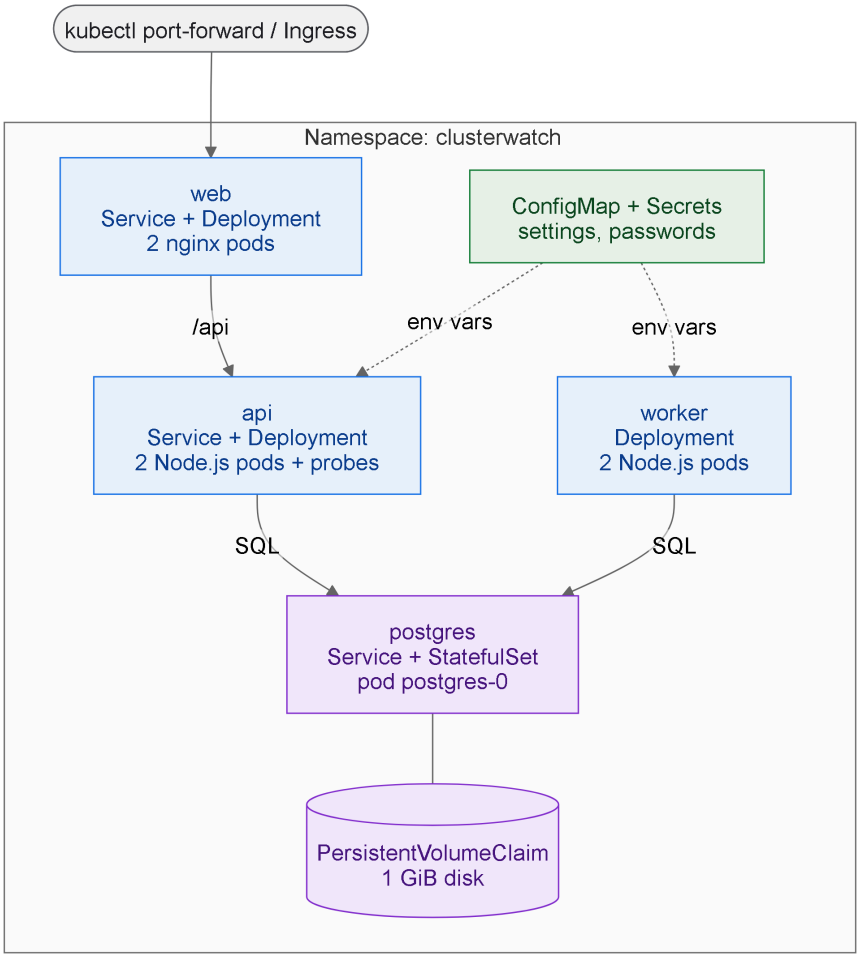
Docker runs containers. But who restarts a container at 3 a.m. when it crashes? Who replaces the old version with the new one without downtime? Who runs it across ten machines?

Kubernetes is the restaurant manager. You don't tell it what to do step by step; you tell it **what you want**: "I want 2 API pods, 2 workers, 2 dashboards, always." Kubernetes keeps comparing what you want with what is running, and fixes the difference forever.

- A pod crashes → it starts a new one.
- A new version is released → it starts new pods, waits until they are **ready**, then removes old ones.
- A pod is not ready → it stops sending it customers but does not fire it.

The building blocks

Kubernetes object	Plain meaning	In ClusterWatch
Pod	One running copy of a container	One API process
Deployment	“Keep N identical pods running, update them safely”	api ×2, worker ×2, web ×2
Service	A fixed name and address that load-balances to ready pods	`api`, `web`, `postgres`
StatefulSet + PVC	A pod with a fixed identity and its own disk	`postgres-0` with 1 GiB of storage
ConfigMap / Secret	Settings and passwords, given to pods as environment variables	Poll interval, SSRF rules, DB password, admin token
Probes	Health questions Kubernetes keeps asking	startup, liveness (`/healthz`), readiness (`/readyz`)
Namespace	A folder for everything that belongs together	`clusterwatch`



Three experiments that prove it works

1. I shipped a broken version on purpose. I set a wrong database password. The new pod crashed in a loop (`CrashLoopBackOff`), but **45 of 45 requests still succeeded**. Kubernetes never removed an old pod because the new one never became ready (`maxUnavailable: 0`). The rollout simply stalled until I rolled it back.

2. I restarted every API pod under load. With a 5-second `preStop` pause: **0 failed requests**. Without it: **23 failed**. When a pod is stopped, Kubernetes sends the stop signal and removes it from the Service at the same moment, and routing takes a few seconds to catch up. The pause keeps the pod serving until it has.

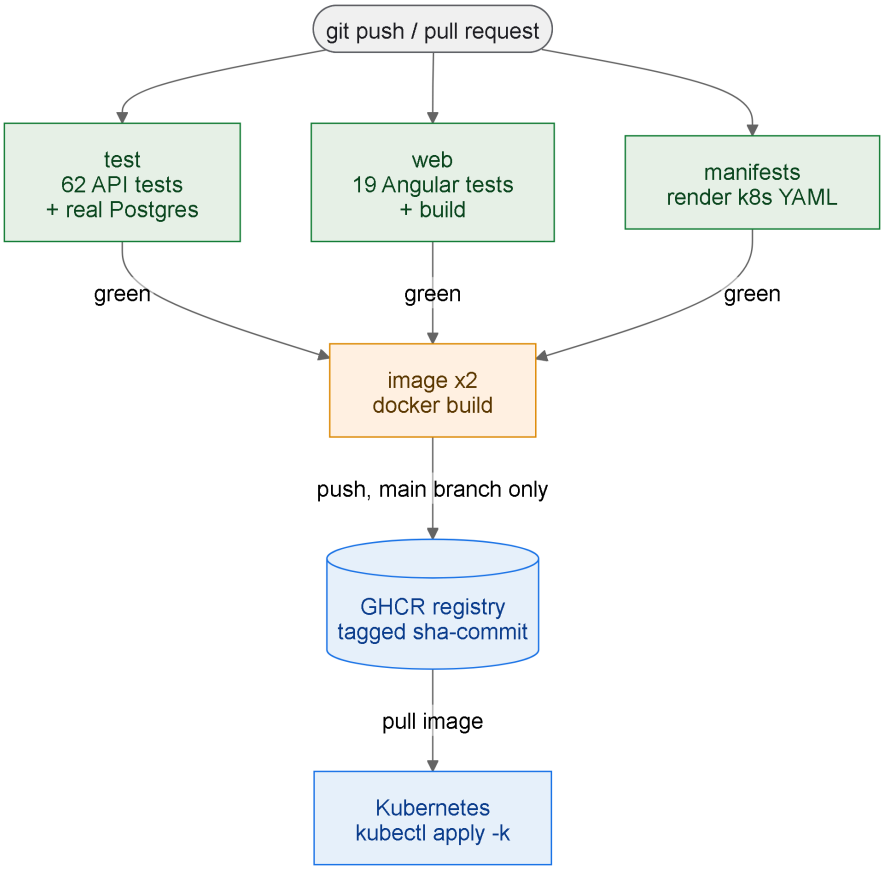
3. I deleted pods by hand. New ones replaced them within seconds. I scaled the workers to 3, then re-applied the files from Git, and it went back to 2. **Git is the source of truth.**

Kubernetes commands

Command	What it does
`kubectl apply -k k8s/`	Create or update everything described in the `k8s/` folder
`kubectl get pods -n clusterwatch`	List pods, their status and restarts
`kubectl describe pod <name> -n clusterwatch`	Full details; the **Events** at the bottom explain failures
`kubectl logs <pod> -n clusterwatch --previous`	Logs of the container that crashed
`kubectl rollout status deploy/api -n clusterwatch`	Wait for a deploy to finish
`kubectl rollout restart deploy/api -n clusterwatch`	Replace pods one by one with zero downtime
`kubectl rollout undo deploy/api -n clusterwatch`	Emergency rollback to the previous version
`kubectl scale deploy/worker --replicas=3 -n clusterwatch`	Change the number of pods
`kubectl port-forward -n clusterwatch svc/web 8081:80`	Reach a Service from your laptop
`kubectl diff -k k8s/`	Show what differs between Git and the cluster

Shipping it

CI/CD: every change is tested before it can ship



On every push, GitHub starts fresh Linux machines that run three checks in parallel. Only if all pass are the two images built and pushed to the GitHub Container Registry, tagged with the commit ID, so it is always clear which code is running. I tested it by opening a pull request with a one-character bug (`>=` changed to `>`): CI failed two tests and refused to build an image.

Going live, safely

Before putting ClusterWatch on a public URL I wrote down what a stranger could do with it, then closed each hole:

Threat	Protection
Strangers change data	Reading is public; changes need an admin token. The API refuses to start in production without one.
The worker is tricked into fetching internal addresses (SSRF)	Private, loopback and cloud-metadata addresses are blocked at connection time; redirects are not followed.
Floods and password guessing	Rate limits per visitor, a stricter limit on failed attempts, a cap of 50 targets.
“What happened at 11:05?”	Every request gets an ID that appears in the response and in the logs.

Then a 25-check **smoke test** ran against the public URL over the internet. All 25 passed.

How to run it

You need Git, Node.js 24 and Docker Desktop (with Kubernetes enabled for option C). Use Git Bash on Windows.

Option A: everything with Docker (fastest)

```
git clone https://github.com/rajanmajithiya/cluster-watch.git
cd cluster-watch

# 1. create your admin token
#    (it stays on your machine: .env is never committed)
echo "ADMIN_TOKEN=$(openssl rand -hex 32)" > .env

# 2. build and start everything:
#    database, API, 2 workers, dashboard, demo services
docker compose up -d --build --wait
docker compose ps
```

Open **http://localhost:8080**, click **Sign in**, and paste the token from `cat .env`. Add a target with URL `http://demo-targets:4000/switch`, then break it and fix it:

```
# an alert opens within about 15 seconds
curl -X POST localhost:4000/switch/down

# it resolves itself about 10 seconds later
curl -X POST localhost:4000/switch/up
```

Stop with `docker compose down`.

Option B: plain Node.js, to see each part

Use four terminals, after stopping Option A (`docker compose down`). Run `npm ci` once first.

```
# terminal 1: PostgreSQL on port 5433
npm run db

# terminal 2: the API on port 3000
export DATABASE_URL=postgres://clusterwatch:clusterwatch@localhost:5433/clusterwatch
npm run dev

# terminal 3: fake services on port 4000
npm run fake-targets

# terminal 4: the worker
export DATABASE_URL=postgres://clusterwatch:clusterwatch@localhost:5433/clusterwatch
npm run worker

# any terminal: all 62 API tests
npm test
```

Option C: Kubernetes

```
# should print docker-desktop
kubectl config current-context

# build both images and copy them into the local cluster
docker build -t clusterwatch:0.9.0 .
docker build -t clusterwatch-web:0.9.0 web
bash scripts/k8s-load-image.sh clusterwatch:0.9.0
bash scripts/k8s-load-image.sh clusterwatch-web:0.9.0

# the admin token goes into a Kubernetes Secret, read from .env
kubectl create namespace clusterwatch
kubectl create secret generic clusterwatch-admin \
  -n clusterwatch --from-env-file=.env

# deploy, then wait for every pod to be Running 1/1 (Ctrl+C to stop watching)
kubectl apply -k k8s/
kubectl get pods -n clusterwatch -w

# open the dashboard at http://localhost:8081
kubectl port-forward -n clusterwatch svc/web 8081:80
```

Remove everything with `kubectl delete namespace clusterwatch`.

The full step-by-step guide, with what each command prints and what to do when it does not, is `docs/RUNBOOK.md` in the repository.

Telling the story in an interview

The 30-second version

> “I built ClusterWatch, a monitoring platform like a mini Pingdom. You register a service’s health URL; Node.js workers check it on a schedule and open one alert after three failures, and resolve it when it recovers. It’s a Node and PostgreSQL backend with an Angular dashboard, packaged with Docker and deployed on Kubernetes, with CI in GitHub Actions. I built it like production and broke it on purpose to learn how it fails.”

The 2-minute walkthrough

Follow the architecture diagram from top to bottom while you talk.

1. **The problem.** Services fail silently; teams hear about it from customers. ClusterWatch notices first.
2. **Node.js.** The API is Express. The workers are separate Node processes because health checking is I/O-heavy, which is exactly what the event loop is good at: 20 checks in flight at once, each with a timeout.
3. **The tricky part, multiple workers.** I used `SELECT … FOR UPDATE SKIP LOCKED`, so workers split the work without ever checking the same target twice. A unique index guarantees one open alert per target.
4. **Docker.** One image for API and worker. A multi-stage build took it from 2 GB to 250 MB, it runs as a non-root user, and Docker Compose starts the whole system with one command.
5. **Kubernetes.** Deployments with two replicas, liveness and readiness probes, and rolling updates with `maxUnavailable: 0`. I proved zero downtime under load: 0 failed requests, versus 23 without a preStop hook.
6. **Shipping.** GitHub Actions runs 81 tests against a real Postgres and pushes images tagged by commit. Before going public I added auth, an SSRF guard and rate limits, and ran a 25-check smoke test over the internet.

Explaining the three technologies simply

If they ask	Say
“Why Node.js?”	“The workload is mostly waiting on the network. Node’s event loop handles thousands of waiting requests on one thread, like one waiter serving many tables who never stands at the kitchen door.”
“What does Docker give you?”	“The app and everything it needs, sealed in one image that runs the same on my laptop, in CI and in the cluster. No more ‘works on my machine’.”
“What does Kubernetes add on top of Docker?”	“Docker runs a container. Kubernetes keeps the system in the state I asked for: it restarts crashed pods, only sends traffic to ready ones, and replaces versions one pod at a time.”
“Docker Compose or Kubernetes?”	“Compose for one machine and local development. Kubernetes for production: many machines, self-healing, rolling updates, scaling.”

Five real bugs I found and fixed

Use these for “tell me about a difficult bug”. Situation, what I did, result.

1. **The wrong password that looked like a slow database.** A bad deploy took 65 seconds per crash because the startup retry treated “wrong password” like “database not up yet”. I separated temporary errors (retry) from permanent ones (fail immediately). Now it crashes in 3 seconds with a clear message.
2. **The 23 failed requests.** A rolling restart under load dropped 23 requests. Kubernetes sends SIGTERM and removes the pod from the Service at the same time. A 5-second `preStop` pause fixed it: 0 failures.
3. **The 500 that should have been a 503.** When I killed the database, the API answered “500 internal error”, which would page on-call for a bug that did not exist. Dependency outages now return 503 with Retry-After.
4. **The button that never came back.** In the Angular dashboard, after a server error the “Add” button stayed disabled forever. New Angular only redraws when a *signal* changes, and my state was a plain variable. I moved it to signals and added a test that fails on the old code.

5. **The monitoring tool that could scan my home network.** A tool that fetches URLs can be abused (SSRF). Before going public I blocked internal addresses at connection time, which also defeats DNS tricks, and stopped following redirects. I verified 8 attack URLs were refused over the public internet.

Questions you should expect

Question	Short answer
Liveness vs readiness?	Liveness failing → Kubernetes restarts the container. Readiness failing → it stops sending traffic but keeps the pod. Never check the database in liveness.
A pod is in CrashLoopBackOff. What do you do?	`kubectl describe` (Events), `kubectl logs --previous`, check the exit code (137 = killed, often out of memory), check config, roll back if needed.
How do you deploy with zero downtime?	2+ replicas, readiness probe, `maxUnavailable: 0`, graceful SIGTERM handling, a preStop pause.
Image vs container?	An image is the read-only package; a container is a running instance of it.
Why multi-stage Docker builds?	Build tools stay in the first stage; only what runs goes in the final image. Smaller, faster, safer.
ConfigMap vs Secret?	Same mechanism; Secrets are for passwords, are only base64-encoded, and need access control and encryption at rest.
How does Node handle many requests on one thread?	The event loop hands I/O to the operating system and runs callbacks when results come back. CPU-heavy work blocks it.
How do you prevent SQL injection?	Parameterised queries (`$1`), so values are never interpreted as SQL.
What is CORS?	A browser rule that stops a page reading responses from another origin. I avoided it by serving the dashboard and API from one origin through nginx.
What stops broken code reaching production?	Required CI checks on the main branch plus code review. CI alone only reports.